

Curs Algoritmi Fundamentali (AF)

Analiza eficienței algoritmilor.

Complexități¹

Conf. dr. Alexandra Băicoianu

Universitatea *Transilvania* din Brașov
Facultatea de Matematică și Informatică

2025

¹Mulțumesc studenților mei pentru corecturile aduse acestui material.

Agenda

- 1 **Eficiența unui algoritm și rolul ei practic**
 - Problemă propusă
 - Sugestii de rezolvare
- 2 **Analiza unui algoritm și eficiența sa**
- 3 **Analiza în cel mai favorabil, cel mai defavorabil și cazul mediu**
- 4 **Analiza în cazul mediu**
 - Importanța cazului cel mai defavorabil
- 5 **Etapele analizei eficienței**
- 6 **Analiza asimptotică: O (Omicron), Ω (Omega), Θ (Theta)**
 - Omicron

Eficiența unui algoritm și rolul ei practic

Ce crede ChatGPT:

Putem asocia **eficiența** unui algoritm cu alegerea unui *traseu* rapid și a unui *rucsac* adecvat pentru o drumeție: ne dorim să ajungem *cât mai repede* la destinație (**timp**) și să *nu cărăm* prea multe lucruri (**memorie**).



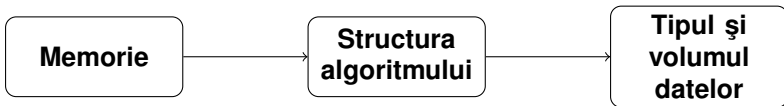
Eficiența unui algoritm și rolul ei practic

Timpul necesar execuției depinde de *numărul operațiilor* ce trebuie executate, iar numărul operațiilor efectuate depinde de *datele de intrare*, care se schimbă de la o execuție la alta.



Eficiența unui algoritm și rolul ei practic

Complexitatea de **memorie** este determinată de *modul de organizare al algoritmului*, respectiv *tipul structurilor de date* utilizate pentru stocare și volumul acestora.



Eficiența unui algoritm și rolul ei practic

Scopul principal al *analizei eficienței algoritmilor* este acela de a determina modul în care **timpul** de execuție crește odată cu creșterea **dimensiunii** problemei.

Astfel, putem compara doi algoritmi în raport cu:

- **cantitatea de memorie necesară**
- **timpul de executare**

Problema propusă

Enunț: Se consideră un tablou cu n elemente având valori din $1, \dots, n$. Tabloul poate avea toate elementele distincte sau poate exista o pereche de elemente cu aceeași valoare (o singură astfel de pereche).

Cerința 1: Să se verifice dacă elementele tabloului sunt toate distincte sau dacă există o pereche de elemente identice.

Cerința 2: Cât este **complexitatea** algoritmului propus?

Sugestii de rezolvare

```
bool areElementeDistingte(const std::vector<int>& tablou) {
    // Sortam tabloul
    std::vector<int> tablouSortat = tablou;
    std::sort(tablouSortat.begin(), tablouSortat.end());

    // Verificam duplicatul
    for (int i = 0; i < tablouSortat.size() - 1; ++i) {
        if (tablouSortat[i] == tablouSortat[i + 1]) {
            return false; // Am gasit o pereche de elemente identice
        }
    }

    return true; // Toate elementele distincte
}
```

Metoda 1: Sortarea elementelor cu `std::sort`² cu complexitatea $O(n \cdot \log n)$ +
 Parcurgerea vectorului sortat $O(n)$
 Complexitatea finală este **$O(n \cdot \log n)$** -> *Eficientă*

²<https://en.cppreference.com/w/cpp/algorithm/sort>

Sugestii de rezolvare

```
bool areElementeDistincteSumaGauss(const std::vector<int>& tablou){
    const int n = tablou.size();

    // Suma Gauss pentru primele n numere este (n * (n + 1)) / 2
    int sumaAsteptata = (n * (n + 1)) / 2;

    // Suma elementelor din tablou
    int sumaTablou = 0;
    for (int elem : tablou) {
        sumaTablou += elem;
    }

    // Dacă suma este aceeași, atunci toate elementele sunt distincte
    return sumaTablou == sumaAsteptata;
}
```

Metoda 2: Calcularea sumei elementelor are complexitatea $O(n)$ + Compararea ei cu suma Gauss $O(1)$
Complexitatea finală este **$O(n)$** -> *Cea mai eficientă*

Analiza unui algoritm

În general, analiza unui algoritm se desfășoară în două etape:

1 Analiza apriorică:

- Estimăm cât de *repede* va lucra algoritmul, luând în considerare **operațiile** pe care le face și **costul** lor relativ.
- Obținem o **expresie matematică** care ne oferă o *limită* pentru timpul de execuție.

2 Testul ulterior:

- Rulăm algoritmul pe diferite **seturi de date** pentru a acoperi *diverse situații*.
- Colectăm datele rezultate și elaborăm *statistici* pentru a înțelege cât de eficient este algoritmul în diverse scenarii (**profilul algoritmului**).

Analiza eficienței unui algoritm

Analiza eficienței algoritmilor înseamnă **estimarea volumului de resurse de calcul** (spațiu de memorie, timp de execuție) necesare execuției algoritmilor.

Analiza eficienței este utilă pentru a **compara algoritmii între ei** și pentru a obține informații despre performanța lor.

Avantajul oferit de analiza teoretică este acela că ea permite evaluarea eficienței indiferent de dimensiunea problemei.

Cum se poate măsura eficiența algoritmilor?

Pentru estimarea timpului de execuție:

- Stabilim un **model de calcul** și alegem ca **unitate de măsură** timpul asociat efectuării unei **operații fundamentale**: *asignare, operații aritmetice, logice, comparații*.
- **Timpul de execuție**, notat $T(n)$, reprezintă *numărul total de operații elementare* realizate de algoritm.
- **Scopul estimării** este să evidențieze legătura dintre *numărul de operații* și *dimensiunea problemei*.

Cum se poate măsura eficiența algoritmilor? (Exemplu)

Să se analizeze numărul de operații pentru **calculul sumei primelor n numere**.

- 1 Scriere de pseudocod/cod C++ pentru rezolvare.
- 2 Să se calculeze numărul de operații pentru $n = 5$. Câte operații avem pentru $n = 10$? Cât este $T(n)$?
- 3 Putem și altă rezolvare? Cât este $T(n)$?

Cum se poate măsura eficiența algoritmilor? (Rezolvare)

Metoda 1: Folosind o structură repetitivă ($O(n)$)

```
int calculateSumLoop(int n) {  
    int sum = 0;  
    for (int i = 1; i <= n; ++i) {  
        sum += i;  
    }  
    return sum;  
}
```

Se efectuează n adunări, deci pentru $n = 5$ vom avea $1 + 1 + 6 + 5 + 5 + (1)$ operații, iar pentru $n = 10$, $1 + 1 + 11 + 10 + 10 + (1)$ operații.

$$\text{Deci } T(n) = 3 \cdot n + 4.$$

Metoda 2: Folosind formula matematică ($O(1)$)

$$S_n = \frac{n \cdot (n + 1)}{2}$$

Se efectuează un singur calcul pentru oricare dimensiune a problemei, deci complexitatea este constantă (*cea mai eficientă abordare*), deci $T(n) = 1$.

Cum se poate măsura eficiența algoritmilor? (Alt exemplu)

Fie $T(n) = n^4 + 2 * n^2 + 10 * n + 500$.

n	n^4	T(n)
1	1	513
3	81	629
5	625	1225
7	2401	3069
10	10000	10800
100	100000000	100021500

Ordinul de creștere expune modalitatea în care crește termenul dominant al timpului de execuție în raport cu dimensiunea problemei.

Analiza în cel mai favorabil și cel mai defavorabil caz

Analiza în cazul cel mai favorabil ne ajută să estimăm cât de rapid poate funcționa un algoritm în cel mai bun scenariu posibil **(limita inferioară)**. Dacă algoritmul are o complexitate slabă și în situația ideală, atunci nu este o alegere bună.

Analiza în cazul cel mai defavorabil furnizează cea mai mare complexitate în raport cu toate datele de intrare de dimensiune n **(limita superioară)**.

Analiza în cazul mediu

Cazul cel mai favorabil și **cazul cel mai defavorabil** reprezintă situații excepționale. **Analiza în cazul mediu** oferă informații despre comportamentul algoritmului în fața unor date de intrare **arbitrare**.

Această analiză se bazează pe cunoașterea distribuției de probabilitate a datelor de intrare. Timpul mediu de execuție reprezintă valoarea medie (**statistic**) a timpilor de execuție pentru diferite instanțe ale datelor de intrare.

Importanța cazului cel mai defavorabil

- **Predictibilitate:** Limita superioară oferă o predictibilitate mai mare asupra comportamentului algoritmului. Dezvoltatorii și proiectanții de algoritmi pot avea o înțelegere clară a performanței maxime, indiferent de datele de intrare.
- **Fiabilitate:** Cazul cel mai defavorabil oferă informații importante despre eficiența algoritmului în situații critice. Este important să se asigure că algoritmul funcționează în mod rezonabil în orice circumstanțe, indiferent de variațiile din datele de intrare.
- **Izolare a ineficiențelor:** Analiza în cazul cel mai defavorabil identifică zonele în care algoritmul poate fi ineficient, ceea ce este esențial pentru îmbunătățirea și optimizarea algoritmilor.

Prin urmare, în dezvoltarea și evaluarea algoritmilor, analiza în cazul cel mai defavorabil primează pentru a asigura un comportament robust și fiabil în toate situațiile posibile.

Care sunt etapele analizei eficienței?

- **Identificarea dimensiunii problemei;**
- **Stabilirea operației dominante:** Identificarea celei mai costisitoare operații din algoritm;
- **Determinarea numărului de execuții ale operației dominante;**
- **Analiza cazurilor:** Dacă numărul de execuții ale operației dominante depinde de proprietățile datelor de intrare, atunci se analizează **cel mai favorabil caz, cel mai defavorabil caz și cazul mediu;**
- **Stabilirea ordinului (clasei) de complexitate.**

Analiza asimptotică: O (Omicron), Ω (Omega), Θ (Theta)

Analiza timpilor de execuție pentru **dimensiuni mici** ale problemei nu permite diferențierea algoritmilor eficienți de cei ineficienți. Diferențele dintre ordinele de creștere devin relevante pe parcursul creșterii dimensiunii problemei.

Analiza asimptotică are ca obiectiv studiul proprietăților timpului de execuție atunci când **dimensiunea problemei tinde către infinit**.

Există trei notații folosite pentru diverse clase de eficiență: O (Omicron), Ω (Omega) și Θ (Theta).

Notăția O (Omicron)

Desemnează marginea asimptotică superioară a unei funcții.

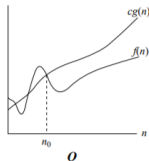
Fie $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$ două funcții care depind de dimensiunea problemei.

Spunem că funcțiile sunt în relația:

$$f(n) = O(g(n))$$

dacă și numai dacă există $n_0 \in \mathbb{N}$, $c > 0$ astfel încât $0 \leq f(n) \leq c \cdot g(n)$, pentru orice $n \geq n_0$.

Pentru valori mari ale lui n , $f(n)$ este mărginită superior de $g(n)$ multiplicată cu o constantă pozitivă:



Notăția O (Omicron) - Exemplu

Se consideră următoarele exemple:

- $f(n) = n^2 + 3n + 7$ și $g(n) = n^2$. Avem $f(n) = O(g(n))$, deoarece există constante $n_0 = 1$ și $c = 11$ astfel încât $0 \leq n^2 + 3n + 7 \leq 11n^2$ pentru orice $n \geq n_0$.
- $f(n) = 2n^3 + 5n^2 + 10$ și $g(n) = n^3$. Avem $f(n) = O(g(n))$, deoarece există constante $n_0 = 1$ și $c = 17$ astfel încât $0 \leq 2n^3 + 5n^2 + 10 \leq 17n^3$ pentru orice $n \geq n_0$.

Notăția O (Omicron) - Proprietăți

- ❶ $O(k) < O(n), \forall k$: Orice constantă este eclipsată de creșterea liniară a lui n .
- ❷ $O(k \cdot f) = O(f), \forall k, f$: O constantă multiplicată cu o funcție are aceeași clasă de complexitate O ca și funcția în sine.
- ❸ $f(n) \in O(f(n))$ (reflexivitate): Orice funcție este în propria sa clasă de complexitate O .
- ❹ $f(n) \in O(g(n)), g(n) \in O(h(n)) \Rightarrow f(n) \in O(h(n))$ (tranzitivitate): Relația O este tranzitivă.
- ❺ $f(n) + g(n) = O(\max\{f(n), g(n)\})$: Suma a două funcții este în O de maximul dintre ele.
- ❻ Dacă $T(n) = a_d \cdot n^d + a_{d-1} \cdot n^{d-1} + \dots + a_1 \cdot n + a_0$, atunci $T(n) \in O(n^k)$ pentru orice $k \geq d$ (funcții polinomiale).
- ❼ Dacă pentru cazul cel mai nefavorabil obținem $T(n) \leq g(n)$, atunci $T(n) \in O(g(n))$: Funcția $T(n)$ este mărginită deasupra de $g(n)$.